```
// functions/api/stripe-webhook.js  —  Cloudflare Pages Function
// It answers at the web address  /api/stripe-webhook
//
// What it does: Stripe calls this address after every successful payment.
// We check the call really came from Stripe, then email the order details
// (items, total, customer, shipping address) to the shop's email address.
//
// Settings (Cloudflare dashboard > your Pages project > Settings > Variables and
//   STRIPE_SECRET_KEY       your Stripe secret key (same one the checkout uses)
//   STRIPE_WEBHOOK_SECRET   the "Signing secret" Stripe shows for this webhook (
//   RESEND_API_KEY          your Resend API key (starts with re_)
//   ORDER_EMAIL             where order emails go, for example lisaabrahamian38@
//   ORDER_FROM              (optional) the "from" line, once you have a verified
//                           for example  Vintage of 4 Seasons <orders@yourdomain

const say = (message, status) =>
  new Response(message, { status, headers: { 'Content-Type': 'text/plain; charset

const money = (cents) => '$' + (Number(cents || 0) / 100).toFixed(2);

// ---- check the call is really from Stripe (signature check, no libraries neede
const toHex = (buf) => [...new Uint8Array(buf)].map((b) => b.toString(16).padStar

async function hmacHex(secret, message) {
  const key = await crypto.subtle.importKey('raw', new TextEncoder().encode(secre
  return toHex(await crypto.subtle.sign('HMAC', key, new TextEncoder().encode(mes
}

function sameText(a, b) {
  if (a.length !== b.length) return false;
  let diff = 0;
  for (let i = 0; i < a.length; i++) diff |= a.charCodeAt(i) ^ b.charCodeAt(i);
  return diff === 0;
}

async function isFromStripe(rawBody, header, secret, nowSeconds = Date.now() / 10
  if (!header) return false;
  let timestamp = '';
  const signatures = [];
  for (const piece of header.split(',')) {
    const [k, v] = piece.split('=');
    if (k === 't') timestamp = v;
    if (k === 'v1' && v) signatures.push(v);
```

```
  }
  if (!timestamp || signatures.length === 0) return false;
  if (Math.abs(nowSeconds - Number(timestamp)) > 300) return false; // older than
  const expected = await hmacHex(secret, timestamp + '.' + rawBody);
  return signatures.some((s) => sameText(s, expected));
}

// ---- write the email (plain text on purpose: nothing a customer typed can run
function buildEmail(session) {
  const customer = session.customer_details || {};
  const collected = (session.collected_information && session.collected_informati
  const a = collected.address || customer.address || {};
  const shipTo = [
    collected.name || customer.name,
    a.line1,
    a.line2,
    [a.city, a.state, a.postal_code].filter(Boolean).join(' '),
    a.country,
  ].filter(Boolean).join('\n');
  const lines = ((session.line_items && session.line_items.data) || [])
    .map((li) => '- ' + li.quantity + ' x ' + li.description + '   ' + money(li.a
    .join('\n');
  const meta = session.metadata || {};
  const who = customer.name || meta.name || 'customer';
  const payment = session.payment_intent
    ? 'https://dashboard.stripe.com/payments/' + session.payment_intent
    : 'https://dashboard.stripe.com/payments';
  const text = [
    'NEW PAID ORDER',
    '',
    'Customer: ' + who,
    'Email: ' + (customer.email || '(none)'),
    'Phone: ' + (customer.phone || meta.phone || '(none)'),
    '',
    'SHIP TO',
    shipTo || '(no address collected)',
    '',
    'ITEMS',
    lines || '(could not read items - check Stripe)',
    '',
    'TOTAL PAID: ' + money(session.amount_total),
    '',
    meta.address ? 'Address typed on the website: ' + meta.address : '',
    'See it in Stripe: ' + payment,
  ].filter((l, i, all) => !(l === '' && all[i - 1] === '')).join('\n');
  return { subject: 'New order ' + money(session.amount_total) + ' - ' + who, tex
```

```
}

export async function onRequest(context) {
  const { request, env } = context;
  if (request.method !== 'POST') return say('Method not allowed', 405);
  for (const name of ['STRIPE_SECRET_KEY', 'STRIPE_WEBHOOK_SECRET', 'RESEND_API_K
    if (!env[name]) return say('Server misconfiguration: ' + name + ' is missing.
  }

  const raw = await request.text();
  if (!(await isFromStripe(raw, request.headers.get('Stripe-Signature'), env.STRI
    return say('Bad signature', 400);
  }
  let event;
  try { event = JSON.parse(raw); } catch (e) { return say('Bad body', 400); }

  // We only care about finished payments; anything else is fine to ignore.
  if (event.type !== 'checkout.session.completed') return say('Ignored', 200);
  const done = event.data && event.data.object;
  if (!done || !done.id) return say('No session', 400);
  if (done.payment_status !== 'paid') return say('Not paid yet', 200);

  // Ask Stripe for the full order (the event itself does not list the items)
  const got = await fetch('https://api.stripe.com/v1/checkout/sessions/' + encode
    headers: { Authorization: 'Bearer ' + env.STRIPE_SECRET_KEY },
  }).catch(() => null);
  if (!got || !got.ok) { console.error('Could not read the order from Stripe'); r
  const session = await got.json();

  const { subject, text } = buildEmail(session);
  const sent = await fetch('https://api.resend.com/emails', {
    method: 'POST',
    headers: { Authorization: 'Bearer ' + env.RESEND_API_KEY, 'Content-Type': 'ap
    body: JSON.stringify({
      from: env.ORDER_FROM || 'Vintage of 4 Seasons <onboarding@resend.dev>',
      to: [env.ORDER_EMAIL],
      subject,
      text,
    }),
  }).catch(() => null);
  // If the email fails we answer 500 so Stripe automatically tries again later.
  if (!sent || !sent.ok) { console.error('Order email failed'); return say('Email
  return say('ok', 200);
}
```